

KASETSART UNIVERSITY
Department of Computer Engineering, Faculty of Engineering

FINAL PROJECT MANUSCRIPT
01219462
Software Engineering for AI-Enabled System

Version 0.1; April 24, 2026

Academic Year 2567 (2024–2025)
Final Submission Deadline: 15 May 2025, 23:59 (via Google Classroom)

Student Name(s)	/
Student ID(s)	/
Project Title	
Role Assignment	
GitHub Repository	https://github.com/

Project Overview

This manuscript serves as the unified final project guide for the course Software Engineering for AI-Enabled System (01219462). It consolidates the Senior Project requirements with the Final Examination Proof-of-Concept (POC) implementation. Students are required to design, implement, and deploy an AI-enabled software system that relates directly to their senior project.

Each team (1–3 members) must clearly define the roles of Data Scientist and Software Engineer across both deliverables. All team members must each submit identical copies of all required documents.

Submission Requirements Summary

Deliverable	Format	Notes
Project Report + Role Assignment (clearly define your team's responsibilities)	PDF file	Cover Part A and B
Jupyter notebook files	.ipynb / shared URL	Cover part B
Video Presentation	Video file	Demo + UI-Model testing
Slide Presentation	PDF file	Highlight key features
Source Code	GitHub repository	README + requirements.txt
Dataset / Sample Data	CSV / ZIP / URL	Or access instructions
MLflow Comparison Screenshots	Images (PNG/JPG)	Part B 1.4
Model Artifact	GitHub repository	Part B 1.7

AI Usage Policy

- **AI as a Feature:** Your project must be built *around* an AI component. This is the core requirement.
- **AI as a Tool:** You may use AI (like ChatGPT, Gemini, Claude) to help write code or fix grammar.
- **The Rule:** You must be able to **explain everything** you submit. If you can't explain how a piece of AI-generated code works, you will lose marks.
- **Disclosure:** Include a short note at the end of your report stating which AI tools you used and what they helped you with.

Bottom line: Use AI to work faster, but you are responsible for the final logic and design.

PART A — AI-Enabled System Design

This part covers the conceptual and architectural design of your AI-enabled software system. You are NOT required to implement these components, focus on thorough documentation and reasoning. **Please submit the entire part as a single PDF file.**

1. Project Definition

Identify the AI-enabled system you will build and justify why AI/ML is necessary.

Task A1.1: Project Title and Motivation

- Provide a clear, descriptive project title (e.g., Fall Detection Watch, Traffic Congestion Predictor)
- Explain the problem domain and target users
- Justify why AI/ML is essential? what makes a rule-based approach insufficient?
- State the expected impact and value delivered to end users

2. System Components Design

Task A2.1: System Objectives

Define system objectives that are Measurable, Achievable, and Communicable (MAC). Include:

- System Goals: what the system must accomplish
- Leading Indicators: early signals that the system is on track
- User Outcomes: tangible benefits experienced by end users

Task A2.2: AI Component Design

- A2.2.1 Identify which specific problem(s) within the system AI is used to solve
- A2.2.2 Define System Goals, Leading Indicators, User Outcomes, and Model Properties for the AI component. For each of these, identify the relevant measures, data collection processes, and operationalization plans. [Activity 3.3]
- A2.2.3 Draw a complete AI and ML Canvas. A draft version of the MLOps Stack Canvas is acceptable.
- A2.2.4 Risk Analysis when the AI component fails:
 - Specify REQ (Requirements), ENV (Environment), and SPEC (Specification)
 - Construct a Fault Tree Analysis (FTA) and identify Minimum Cut Sets
 - Propose strategies to mitigate each identified failure risk

Task A2.3: User Interaction Design (Intelligence Experiences)

- A2.3.1 Design the User Interaction Experience. Describe how to present intelligence to the user (via automation, prompting, annotation, or a hybrid approach) and provide a brief explanation.
- A2.3.2 Specify where the AI model will live (Cloud / Edge / Embedded Device) with justification
- A2.3.3 Beyond model accuracy, identify additional considerations relevant to your system objectives. Include justifications for factors such as prediction speed, model size, and latency
- A2.3.4 Explain how to compose models if multiple AI components are involved

Task A2.4: Feedback Collection and Model Monitoring

- Design the mechanism for collecting user feedback after predictions
- Define metrics and tools to monitor model performance in production
- Describe how feedback loops feed into model retraining pipelines

PART B — Proof-of-Concept ML Model Implementation

Implement a proof-of-concept (POC) ML model and integrate it with a software system. Each task below maps to a graded deliverable. All code must be uploaded to GitHub with proper documentation.

Task B-1: Input Data Description and Exploration

- Describe features and data characteristics: data types, ranges, distributions, and example records
- Large-scale training data: if your dataset is very large, explain your strategy (e.g., sampling, distributed training, data streaming)
- Sensitive features and model fairness: identify any sensitive attributes in your data and describe how you handle them to ensure fairness

Required file: A Jupyter notebook (or compatible file) containing saved results that demonstrate the data exploration process.

Task B-2: Model Training Implementation

Write code to train a machine learning model. Your implementation must include:

- Data preprocessing functions (normalization, encoding, handling missing values)
- Model architecture definition and hyperparameter specification
- Training loop with appropriate validation methodology (e.g., k-fold, hold-out)
- Results analysis: confusion matrix, classification report, or equivalent

Required file: A Jupyter notebook (or compatible file) containing saved results that demonstrate the entire model training and evaluation process.

Task B-3: Model Fairness Analysis (if applicable)

Analyze your model for potential biases and fairness issues:

- Identify potential sources of bias in your training data (sensitive features)
- Implement at least one fairness metric calculation (e.g., demographic parity, equalized odds)
- Discuss approaches to mitigate any detected bias

Task B-4: Model Versioning and Experimentation

Compare at least three versions of your model using MLflow Tracking UI or a similar tool:

- Track different hyperparameters, architectures, or data preprocessing approaches across runs
- Document performance differences quantitatively in a comparison table
- Justify your final model selection based on the experimental evidence

Required file: Screenshots of the MLflow Tracking UI comparison (PNG/JPG).

Task B-5: Model Explainability

Implement and demonstrate techniques for model interpretability:

- Apply appropriate explainability methods for your model type (e.g., feature importance)
- Visualize feature importance or decision boundaries
- Interpret the results in terms of your application domain

Required file: A Jupyter notebook (or compatible file) containing saved results that demonstrate the model interpretability.

Task B-6: Prediction Reasoning

Design a mechanism to provide human-understandable reasoning for individual predictions:

- Implement code that explains model decisions (e.g., using SHAP values or LIME)
- Show example output for at least three different test cases
- Ensure the reasoning output is interpretable to a non-technical end user

Required file: A Jupyter notebook (or compatible file) containing saved results that demonstrate the individual r.

Task B-7: Model Deployment as a Service

Deploy your trained model as an API service using MLflow Models or equivalent:

- Implement a RESTful API using a framework of your choice (e.g., FastAPI, Flask, MLflow Serving)
- Document all API endpoints, expected input formats, and output formats
- Demonstrate validation of the deployed model's functionality
- Discuss scalability considerations (horizontal scaling, batching, caching)

Required files: MLmodel file and model.pkl (or equivalent serialized model artifact).

PART C — UI-Model Interface

Design, document, and test the full interface between your user-facing application and your deployed AI model service.

Task C-1: UI Design

Design the user-facing application that interacts with your AI model:

- Provide mockups (sample screens/wireframes) illustrating how AI predictions are presented to users
- Show how users can provide feedback on AI predictions to support continuous improvement
- Address the Intelligence Experience: how the AI's uncertainty or confidence is communicated

Task C-2: UI-Model Interface Design

Design the full interface contract between UI and model service:

- Document the complete request payload structure sent from the UI (JSON schema with field descriptions and types)
- Document the response payload structure returned by the model (including confidence scores, prediction labels, etc.)
- Include error handling specifications: HTTP status codes and error message formats
- Provide a sequence diagram illustrating the end-to-end interaction flow (UI → API → Model → UI)

Task C-3: Interface Testing and Implementation

Test and validate your UI-Model interface using one of the following approaches:

- Postman: Create a collection of test requests and document results for at least three scenarios
- Web Application or Mobile App: Implement a simple frontend that integrates with your deployed API

In either case, you must:

- Demonstrate at least three different test scenarios with distinct inputs
- Document any challenges encountered during integration and how you resolved them
- Include screenshots or video demonstrating successful API calls

Evaluation Criteria

Your final project will be evaluated across the following dimensions:

Criterion	Description	Weight
Technical Correctness & Completeness	All required tasks implemented; models trained and deployed correctly	30%
Code Quality & Documentation	Clean, well-commented code; clear README and requirements.txt	20%
Thoughtfulness of Design Decisions	Justified choices for model type, deployment, evaluation metrics	20%
Critical Analysis & Reflection	Honest discussion of limitations, trade-offs, and failure modes	15%
Innovation & Creativity	Novel approaches, creative UI/UX, extra features beyond the minimum	15%